

MSME CREDIT INTELLIGENCE PLATFORM

How the synthetic banking data became a credit-risk model,
live scoring service and Customer 360 foundation

Technical modelling guide

Data preparation • Point-in-time joins • Feature engineering •
Model evaluation • Deployment

1. Executive summary

The MVP follows a traceable path from operational-style CSV files to an auditable probability-of-default score.

13	2,380	49	11.89%
source CSVs	eligible training loans	model features	training default rate

- The 13 CSV files represent a small banking ecosystem: customers, accounts, transactions, applications, bureau data, assessments, decisions, loans, repayment behaviour, alerts, users and audit activity.
- Not every operational dataset was used to train the model. Training deliberately excluded post-decision and administrative information that would create target leakage.
- The training grain is one historical loan application that produced a loan and had an observable 12-month performance outcome.
- All behavioural features were calculated as they would have been known on the application date. Transactions were limited to the preceding 180 days and future accounts or bureau reports were excluded.
- Three classifiers were compared. Random Forest became the champion because it produced the strongest validation precision-recall performance while meeting the chosen recall objective.
- The champion was deployed behind a point-in-time PostgreSQL scoring view and a FastAPI service. Every score can be stored as a new version in `credit.credit_assessments`.

Core modelling principle: predict future loan default using only information available when the lending decision was made. Any field created after that decision is prohibited from the model input.

Document map

2. The 13 source datasets
3. Data generation, loading and cleaning
4. How the datasets were combined
5. Target construction and leakage prevention
6. Feature engineering
7. Training and model comparison
8. Evaluation and interpretation
9. Production scoring
10. Customer 360
11. Reproducibility, governance and limitations
12. Project placement

2. The 13 source datasets

The regenerated dataset was loaded into three PostgreSQL schemas: banking, credit and security.

#	CSV / PostgreSQL table	Rows	Role in platform	Model use
1	users / security.users	4	Analysts, managers and system users	No - governance
2	customers / banking.customers	5,000	MSME identity, industry, age, declared turnover	Yes
3	accounts / banking.accounts	10,046	Deposit accounts and relationship history	Yes
4	account_transactions / banking.account_transactions	400,000	Cash-flow behaviour and balances	Yes
5	loan_applications / credit.loan_applications	5,500	Requested facility and declared affordability	Yes - anchor
6	credit_bureau_reports / banking.credit_bureau_reports	5,500	External credit obligations and adverse history	Yes
7	credit_assessments / credit.credit_assessments	5,500	Historical or generated score records	No - post-score
8	credit_decisions / credit.credit_decisions	5,500	Analyst/manager/final decisions	No - leakage
9	loans / banking.loans	3,126	Disbursed exposure and performance outcome	Yes - label/cohort
10	repayment_schedule / banking.repayment_schedule	54,969	Expected instalments and delinquency	Outcome validation / 360
11	loan_payments / banking.loan_payments	40,877	Actual repayment behaviour	Outcome validation / 360
12	risk_alerts / credit.risk_alerts	287	Early-warning cases	No - operational output
13	audit_logs / security.audit_logs	1,037	Traceability of platform activity	No - governance

Why only a subset trained the model: decisions, existing assessments and risk alerts already contain conclusions about risk. Feeding them into training would make validation look unrealistically strong and would fail on a genuinely new application.

3. Data generation, loading and cleaning

Synthetic data was generated to preserve realistic keys, dates, statuses and financial relationships without exposing real customer information.

3.1 Generation and relational consistency

- Customers were generated first. Accounts and applications referenced valid `customer_id` values.
- Transactions referenced valid `account_id` values; bureau reports referenced applications; loans referenced approved applications and loan accounts.
- Schedules and payments referenced valid `loan_id` values. Assessments, decisions and alerts referenced the relevant application or customer entities.
- Primary keys, foreign keys, unique constraints and CHECK constraints in PostgreSQL acted as the final quality gate.

3.2 Corrections made before loading

Area	Cleaning action	Reason
Enumerated statuses	Mapped generated values to permitted customer, KYC, application, decision, payment, schedule and alert codes.	Meet database domain rules.
Financial amounts	Kept amounts non-negative where required and reconciled payment components with total payment.	Preserve accounting identities.
Repayment schedules	Enforced <code>total_due = principal + interest + fees</code> and <code>outstanding = total_due - paid</code> .	Prevent internally inconsistent arrears.
Dates	Ensured incorporation precedes relationship start; account/loan dates are chronologically valid.	Prevent impossible histories.
Identifiers	Created missing users and relationship-manager references; added unique loan accounts where required.	Preserve referential integrity.
Missing values	Retained meaningful nulls in source data; handled them inside the model pipeline rather than deleting whole loans.	Avoid unnecessary sample loss.
Duplicates	Database keys rejected duplicate entity IDs; unique application/version and loan-account constraints were respected.	Keep one authoritative record per grain.

3.3 Load method

- The loader validated that every required CSV existed and compared CSV headers with PostgreSQL columns.
- Old synthetic rows were truncated in dependency-safe order, then the 13 files were bulk loaded in one transaction.
- If any table failed, PostgreSQL rolled back the transaction. The successful run committed all datasets together.

4. How the datasets were combined

The model dataset is not a blind merge of every CSV. It is a controlled, point-in-time relational feature view.

Historical loan	Loan application	Customer
Defines eligible cohort and observed outcome	Defines scoring date and requested facility	Adds business profile
Accounts	180-day transactions	Latest bureau report
Relationship depth before application	Cash-flow behaviour before application	External obligations known at application

Join / aggregation	Key and time condition	Output grain
Loans → applications	loan.application_id = application.application_id	One eligible historical loan/application
Applications → customers	application.customer_id = customer.customer_id	Business characteristics
Applications → accounts	customer_id; account.open_date ≤ application_date	Counts and oldest-account age
Accounts → transactions	account_id; application_date - 180 days ≤ value_date < application_date; exclude reversals	One 180-day behavioural aggregate
Applications → bureau	application_id; latest report_date ≤ application_date	One pre-decision bureau snapshot
Loans → outcome	Observed loan performance during the defined 12-month outcome window	default_12m label

Resulting analytical dataset

- 2,380 eligible historical loan records.
- 54 columns: four identifiers/context fields (loan_id, application_id, customer_id and application_date), 49 predictive features and one target (default_12m).
- 2,282 unique customers, meaning a small number had more than one eligible historical borrowing event.
- 283 defaults and 2,097 non-defaults, producing an 11.89% default rate.

5. Target construction and leakage prevention

A model can only be trusted if the target represents future behaviour and the predictors represent information available before that future occurred.

5.1 Target: default_12m

For the MVP, a historical loan was labelled 1 (default) when its observed performance crossed the defined serious-delinquency/default condition, including 90+ days past due or loss classification. Otherwise it was labelled 0 (non-default). The outcome is attached after the feature snapshot is formed.

Non-default	2,097	88.11%
Default	283	11.89%

5.2 Leakage controls

Potential leakage	Control implemented
Transactions after application	Strictly required <code>value_date < application_date</code> .
Accounts opened after application	Required <code>open_date ≤ application_date</code> .
Later bureau information	Selected the latest report no later than application date.
Existing assessment score	<code>credit_assessments</code> excluded from model inputs.
Human/final decision	<code>credit_decisions</code> excluded from model inputs.
Risk alert raised after deterioration	<code>risk_alerts</code> excluded from model inputs.
Repayment outcome	Used only for label creation/validation, never as a predictor.

The live view `credit.ml_scoring_features_v1` applies the same point-in-time rules and exposes 3 identifiers plus exactly the 49 trained feature columns. Its compatibility test scored 25 rows successfully, with probabilities ranging from 0.0393 to 0.8178.

6. Feature engineering

Raw banking records were converted into stable application-level signals that a classifier can consume.

Feature family	Examples	Business interpretation
Business profile	business_type, industry_code, county_code, employee_count, turnover	Size, sector and operating context
Relationship history	business_age_months, relationship_age_months, account_count, oldest_account_age_months	Operating maturity and depth with bank
Loan request	product, purpose, requested amount, requested term, channel	Structure and intent of borrowing
Declared affordability	monthly revenue, expenses, existing debt, operating margin	Ability to absorb additional repayment
Exposure ratios	amount/turnover, debt/revenue, collateral coverage	Request size relative to capacity/security
Transaction activity	180-day counts, totals, averages, standard deviation	Scale and stability of observed operations
Liquidity behaviour	average/min/max balance, net cash flow, debit/credit ratio, cash count	Liquidity pressure and cash dependency
Bureau profile	bureau score, active/closed facilities, outstanding balance, monthly obligation	External leverage and credit standing
Adverse credit	prior max DPD, delinquent/written-off facilities, legal cases, enquiries, adverse flag	Evidence of distress or credit shopping

6.1 Derived feature examples

Feature	MVP calculation
requested_amount_to_turnover	$\text{requested_amount} / \text{annual_turnover_declared}$
declared_operating_margin	$(\text{monthly_revenue} - \text{monthly_expenses}) / \text{monthly_revenue}$
existing_debt_to_revenue	$\text{existing_monthly_debt} / \text{monthly_revenue}$
collateral_coverage_ratio	$\text{collateral_value} / \text{requested_amount}$
net_cash_flow_180d	$\text{total_credits_180d} - \text{total_debits_180d}$
debit_to_credit_ratio_180d	$\text{total_debits_180d} / \text{total_credits_180d}$

6.2 Missing values and types

- Identifiers and the target were separated before modelling.
- Numeric columns were processed through an imputation pipeline, using robust central-value replacement where values were absent.
- Categorical columns were imputed and encoded with unknown-category handling so future applications do not fail when a new valid category appears.
- The processed CSV contained only 36 missing values in debit_to_credit_ratio_180d, caused by applications with no qualifying credits; the model pipeline handled them rather than deleting those applications.
- The complete preprocessing and classifier were stored together in one joblib pipeline, preventing training-serving transformation mismatch.

7. Training and model comparison

The workflow separated model development from final testing and compared interpretable and nonlinear classifiers.

TRAIN	VALIDATION	TEST
1,424 rows	475 rows	481 rows
169 defaults	58 defaults	56 defaults
Fit model parameters	Compare models and select threshold	One-time unbiased champion evaluation

7.1 Candidate models

Model	Why included	Main trade-off
Logistic Regression	Transparent linear baseline; useful for governance and challenger monitoring.	May miss nonlinear interactions.
Random Forest	Captures nonlinearities and interactions; robust on mixed tabular features.	Less directly interpretable; requires controlled reason-code layer.
Gradient Boosting	Strong tabular-data candidate that learns sequential corrections.	More sensitive to tuning and synthetic-signal artefacts.

7.2 Validation results

Model	ROC-AUC	PR-AUC	Threshold	Precision	Recall	F1	FP	FN
Random Forest	0.8802	0.4792	0.3640	0.4196	0.8103	0.5529	65	11
Logistic Regression	0.8807	0.4667	0.4577	0.3643	0.8103	0.5027	82	11
Gradient Boosting	0.8591	0.4411	0.0497	0.3243	0.8276	0.4660	100	10

Champion choice: Random Forest. It had the highest validation PR-AUC and F1 among the compared models while detecting about 81% of defaults at the validation-selected threshold of 0.363984.

Why PR-AUC mattered

Only 11.89% of the training observations were defaults. With this imbalance, accuracy and ROC-AUC can look strong even when the model produces too many false alerts. Precision-recall metrics focus more directly on performance for the minority default class.

8. Champion evaluation and interpretation

The untouched test set measured expected behaviour after model and threshold selection.

Metric	Test value	Interpretation
ROC-AUC	0.8820	Strong overall ranking of defaults above non-defaults.
PR-AUC	0.4302	Meaningful minority-class lift above the 11.6% test prevalence.
Precision	0.3932	About 39% of model-flagged cases were actual defaults.
Recall	0.8214	Detected 46 of 56 defaults; missed 10.
F1	0.5318	Balanced precision-recall summary.
Accuracy	0.8316	Overall correct classification rate; not used alone.
Brier score	0.0896	Probability error; lower is better.

	Predicted non-default	Predicted high-risk
Actual non-default	TN = 354	FP = 71
Actual default	FN = 10	TP = 46

8.1 Observed feature relationships

Feature	Non-default mean	Default mean	Observed direction
Bureau score	702.76	594.93	Lower score associated with default
Previous maximum DPD	3.30	11.77	Higher prior delinquency associated with default
Delinquent facilities	0.0029	0.0495	Higher delinquency associated with default
Declared operating margin	0.3358	0.3177	Slightly lower margin among defaults
Collateral coverage	0.9205	0.9042	Slightly lower coverage among defaults

Sparse indicators require caution: adverse listings appeared in only 4 records, written-off facilities in 1, and legal_cases was always zero. These are better treated as explicit policy rules until real data provides enough variation for reliable statistical learning.

9. Production scoring design

The deployed scoring path reproduces the training feature contract and records an auditable assessment version.

Stage	Implemented component	Responsibility
Feature retrieval	credit.ml_scoring_features_v1	Produces one application row with 49 model inputs using point-in-time rules.
Compatibility check	validate_scoring_view.py	Compares column names/order to model metadata and runs predict_proba smoke test.
Preprocessing + model	champion_credit_risk_pipeline.joblib	Applies fitted transformations and Random Forest without retraining.
Decision layer	CreditRiskService	Maps probability to risk band, credit score, reason codes and policy recommendation.
Persistence	credit.credit_assessments	Stores immutable versioned assessment, feature snapshot and policy results.
API	FastAPI credit-risk endpoints	Scores an application, returns latest assessment and exposes model metadata.

9.1 Risk bands

Probability of default	Risk band	System recommendation
< 0.10	VERY_LOW	RECOMMEND_APPROVAL
0.10 to < 0.20	LOW	RECOMMEND_APPROVAL
0.20 to < 0.363984	MEDIUM	REFER_FOR_APPROVAL
0.363984 to < 0.60	HIGH	REFER_FOR_ENHANCED_REVIEW
≥ 0.60	VERY_HIGH	RECOMMEND_DECLINE

9.2 Model output versus lending decision

- The probability and risk band are model outputs; they rank expected credit risk.
- Policy rules can override or escalate the system recommendation for adverse listings, write-offs, legal cases or insufficient information.
- Recommended repayment capacity and limit are transparent MVP affordability calculations, not outputs learned by the Random Forest.
- The final lending decision remains a human or governed credit-policy decision. Model output is decision support, not an automatic legal approval.

10. Customer 360: datasets and purpose

Customer 360 is a multi-domain operational view. It is broader than the model dataset because it supports analysts, servicing and monitoring as well as origination.

Customer 360 section	Datasets used	Information shown
Identity and profile	customers	Business name, registration, type, industry, county, age, turnover, KYC/AML and relationship manager.
Banking relationship	accounts	Products held, status, balances, overdraft and relationship age.
Cash-flow analytics	account_transactions + accounts	Credits/debits, net flow, balance trend, cash intensity, channels and transaction categories.
Application history	loan_applications	Requested amounts, purposes, terms, channels, statuses and declared affordability.
Risk history	credit_assessments	Score versions, PD, bands, reason factors, feature snapshots and recommendations.
Decision history	credit_decisions + users	Analyst/manager/final decisions, overrides and decision ownership.
Credit exposure	loans	Original and outstanding principal, arrears, DPD, classification and maturity.
Repayment behaviour	repayment_schedule + loan_payments	Expected vs actual instalments, missed/late payments and allocation of principal/interest/fees.
External credit	credit_bureau_reports	Score, obligations, facilities, enquiries, delinquency, write-offs and adverse listings.
Early warning	risk_alerts	Open alerts, severity, investigation status and resolution.
Audit and accountability	audit_logs + users	Who viewed, scored or changed a record and when.

10.1 Customer 360 aggregation approach

- The customer table is the root entity, keyed by customer_id.
- Accounts, transactions, applications, bureau records, loans and alerts are grouped or filtered by customer_id, directly or through their parent application/account/loan.
- One-to-many domains are summarized for the overview and exposed as detailed timelines or paginated lists when an analyst drills down.
- The latest credit assessment supplies the current score, while previous versions provide the risk-history timeline.
- Customer 360 may display post-application data because its purpose is operational understanding. The predictive model view must still enforce point-in-time restrictions.

Important distinction: Customer 360 can use all relevant current and historical data for analyst context. Model scoring uses only the approved 49 pre-decision predictors. Keeping these layers separate protects model validity.

11. Reproducibility, governance and limitations

The MVP demonstrates the complete architecture, but synthetic-data performance is evidence of pipeline correctness rather than proof of real-world credit performance.

11.1 Reproducible artifacts

Artifact	Purpose
credit_risk_training_dataset.csv	Frozen analytical dataset used for modelling.
training_dataset_quality_report.json	Counts, defaults, missingness and quality diagnostics.
champion_credit_risk_pipeline.joblib	Fitted preprocessing and Random Forest pipeline.
champion_metadata.json	Feature contract, categorical fields, threshold and model metadata.
credit.ml_training_dataset_v1	Rebuildable historical training feature/label view.
credit.ml_scoring_features_v1	Live point-in-time inference features.
credit.credit_assessments	Versioned predictions, reasons, policies and feature snapshots.

11.2 Current limitations

- All observations are synthetic. Real deployment requires retraining and independent validation using governed historical bank data.
- Some adverse variables are extremely sparse or constant, so their learned importance is not reliable.
- The model needs probability calibration, stability analysis and segment-level fairness testing on real data.
- The present threshold prioritizes default recall and therefore generates false positives. The bank must set the operating threshold using its loss appetite, review capacity and approval strategy.
- Affordability and recommended-limit formulas are MVP policy assumptions and must be replaced with approved credit-policy calculations.
- Reason codes are deterministic business explanations around the score; they are not yet local SHAP explanations of the Random Forest prediction.

11.3 Recommended production validation

- Define the observation window, performance window, default definition and cure/write-off treatment with Credit Risk.
- Use an out-of-time test set and, where possible, an out-of-sample geographic or product validation set.
- Measure ROC-AUC, PR-AUC, KS, calibration, Brier score, approval rate, bad rate, expected loss and stability index.
- Review performance by sector, county, business type, product, gender/ownership where legally permitted, and thin-file status.
- Establish model registry, approvals, access controls, monitoring, override reporting, drift alerts and retraining triggers.

12. Adding this guide to the project

Keep technical documentation version-controlled beside the implementation so future developers can understand the model contract.

Recommended project path

```
docs/modeling/MSME_Credit_Intelligence_Modeling_Guide.pdf
```

Recommended repository structure

```
msme-credit-intelligence/  
├── backend/  
│   ├── app/  
│   └── ml/  
├── artifacts/  
├── build_training_dataset.py  
├── compare_credit_models.py  
├── train_baseline_model.py  
├── data/  
│   ├── generated/  
│   └── processed/  
├── docs/  
├── modeling/  
├── MSME_Credit_Intelligence_Modeling_Guide.pdf  
└── README.md
```

Suggested README entry

```
### Model documentation
```

The end-to-end data preparation, feature engineering, model selection, evaluation, live scoring and Customer 360 design are documented in:

```
`docs/modeling/MSME_Credit_Intelligence_Modeling_Guide.pdf`
```

Implementation checkpoint

- Synthetic relational data: complete and loaded.
- Training dataset and quality report: complete.
- Baseline and candidate-model comparison: complete.
- Champion Random Forest and threshold: selected and saved.
- Live scoring feature view: validated against 49-feature model contract.
- FastAPI single-application scoring layer: prepared for end-to-end testing.
- Next delivery phase: portfolio APIs and Angular risk/customer dashboards.

End of guide